- **Use the platform as an advantage.** Touch-screen navigation is not intuitive and must be learned by all new users. This learning task will be easier if the user interface designers adhere to standards that have been set for the platform.
- **Make scrollbars and selection highlighting more salient.** Scrollbars are often hard to locate on touch devices because they are too small. Make sure that menu or icon borders are wide enough for color changes to catch the users' attention. When color coding is used, make sure there is sufficient contrast between foreground and background colors to allow them to be distinguishable by any color-blind users.
- **Increase discoverability of advanced functionality.** Hot keys and other shortcuts are sometimes included in mobile products to allow experienced users to complete their tasks more quickly. You can increase the discoverability of features like these by including visual design clues in the user interface.
- **Use clear and consistent labels.** Widget labels should be recognized by all app users, regardless of standards used by specific platforms. Use abbreviations cautiously and avoid them if possible.
- **Clever icons should never be developed at the expense of user understanding.** Icons often only make sense to their designers. Users must be able to learn their meaning quickly. It is hard to guarantee that icons are meaningful across all languages and user groups. A good strategy to enhance recognition is to add a text label beneath a novel icon.
- **Support user expectations for personalization.** Mobile device users expect to be able to personalize everything. At the very least, developers should try to allow users to set their location (or detect it automatically) and select content options that may be available at that location. It is important to indicate to users what features can be personalized and how users can personalize them.
- **Long scrolling forms trump multiple screens on mobile devices.** Experienced mobile device users want all information on a single input screen even if this requires scrolling. Novice users often become experienced quickly and will grow tired of multiple input screens.

Developing native applications for multiple device platforms can be costly and time consuming. Development costs can be reduced by using technologies familiar to Web developers (e.g., JavaScript, CSS, and HTML) to create mobile products that will be accessed using a Web browser on a mobile device.

There are no guarantees that a desktop program or a WebApp can be easily adapted for implementation as a mobile product. However, many of the agile software engineering practices (Chapter 3) used to create desktop computer applications can be used to create stand-alone apps or mobile client software, and many of the practices used to create quality WebApps apply to the creation of Web services used by mobile products.

The most important architectural design decision is often whether to build a thin or fat client. The model-view-controller (MVC) architecture (Section 13.3) is commonly used in mobile products. Because the mobile architecture has a strong influence on navigation, the decisions made during this design action will influence work conducted during navigation design. The architectural design must take device resources

into account (storage, processor speed, and network connectivity). The design should include provisions for discoverable services and movable devices.

Usability testing and deployment testing take place during each prototype development cycle. Code reviews that focus on security issues should be included as part of the implementation activities. These code reviews should be based on the appropriate security objectives and threats identified in the system design activities. Security testing is a routine part of system testing (Chapter 21).

13.9 SUMMARY

The quality of a mobile product—defined in terms of functionality, reliability, usability, efficiency, security, maintainability, scalability, and portability—is introduced during design. A good mobile product should be based on the following design goals: simplicity, ubiquity, personalization, flexibility, and localization.

Interface design describes the structure and organization of the user interface and includes a representation of screen layout, a definition of the modes of interaction, and a description of navigation mechanisms. In addition, the interface for a good mobile product will promote the brand signature and focus on its targeted device platform(s). A set of core user stories is used to trim unnecessary features from the app to manage its resource requirements. Context-aware devices make use of discoverable services to help personalize the user experience.

Content design is critically important and takes the screen and other limitations of mobile devices into account. Aesthetic design, also called graphic design, describes the "look and feel" of the mobile product and includes color schemes, graphic layout, the use of graphics, and related aesthetic decisions. Aesthetic design must also take device limitations into account.

Architecture design identifies the overall hypermedia structure for the mobile product and encompasses both content architecture and mobile architecture. It is critical to determine how much of the mobile functionality will reside on the mobile device and how much will be provided by Web or cloud services.

Navigation design represents the navigational flow between content objects and for all mobile functions. Navigation syntax is defined by the widgets available on the targeted mobile device(s), and the semantics are often determined by the mobile platform. Content chunking must take intermittent service interruptions and user demands for fast performance into account.

Component design develops the detailed processing logic required to implement the components that are used to build a complete MobileApp function. The design techniques described in Chapter 12 may be applicable for the engineering of mobile components.

PROBLEMS AND POINTS TO PONDER

13.1. Explain why deciding to develop a MobileApp for several devices can be a costly design decision. Is there a way to mitigate the risks of supporting the wrong platform?

13.2. In this chapter we listed many quality attributes for mobile products. Select the three that you believe are most important, and make an argument that explains why each should be emphasized in mobile design work.

13.3. You are a MobileApp designer for *Project Planning Corporation*, a company that builds productivity software. You want to implement the equivalent of a digital three-ring binder that allows tablet users to organize and categorize electronic documents of several types under user-defined tabs. For example, a kitchen remodeling project might require a pdf catalog, a jpg or layout drawing, an MS Word proposal, and an Excel spreadsheet stored under a Cabinetry tab. Once defined, the binder and its tab content can be stored either on the tablet or on some cloud storage. The application needs to provide five key functions: binder and tab definition, digital document acquisition from a Web location or the device, binder management functions, page display functions, and a notes function to allow a Post-it note to be added to any page. Develop an interface design for the three-ring application, and implement it as a paper prototype.

13.4. What is the most aesthetically pleasing MobileApp you have ever used and why?

13.5. Create user stories for the three-ring application described in Problem 13.3.

13.6. What might be considered to make the three-ring application a context-aware MobileApp?

13.7. Reconsidering the *ProjectPlanning* three-ring application described in Problem 13.3, select a development platform for the first working prototype. Discuss why you made the choice.

13.8. Do a bit of additional research on the MVC architecture and decide whether it would be an appropriate MobileApp architecture for the three-ring discussed in Problem 13.3.

13.9. Describe three context-aware features that would be desirable to add to a *SafeHome* MobileApp.

13.10. You are a WebApp designer for FutureLearning Corporation, a distance learning company. You intend to implement an Internet-based "learning engine" that will enable you to deliver course content to a student. The learning engine provides the basic infrastructure for delivering learning content on any subject (content designers will prepare appropriate content). Develop a prototype interface design for the learning engine.

Design element: Quick Look icon magnifying glass: © Roger Pressman

CHAPTER

14 PATTERN-BASED DESIGN

Each of us has encountered a design problem and silently thought: *I wonder if anyone has developed a solution for this?* The answer is almost always *yes!* The problem is finding the solution; ensuring that it does, in fact, fit the problem you've encountered; understanding the constraints that may restrict the ways in which the solution is applied; and finally, translating the proposed solution into your design environment.

KEY CONCEPTS

anti-patterns 302
architectural patterns 299
behavioral patterns 292
component-level design patterns 300
creational patterns 292
design mistakes 298
design patterns 290
frameworks 293
kinds of patterns 291
machine learning 294
pattern languages 297
pattern-organizing table 298
structural patterns 292
system of forces 290
user interface design patterns 304



QUICK LOOK

What is it? Pattern-based design creates a new application by finding a set of proven solutions to a clearly delineated set of problems. Each problem and its solution are described by a design pattern that has been cataloged and vetted by other software engineers who have encountered the problem and implemented the solution while designing other applications.

Who does it? A software engineer examines each problem encountered for a new application and then attempts to find a relevant solution by searching one or more pattern repositories.

Why is it important? Have you ever heard the phrase "reinventing the wheel"? It happens all the time in software development, and it's a waste of time and energy. By using existing design patterns, you can acquire a proven solution for a specific problem. As each pattern is applied, solutions are integrated and the application to be built moves closer to a complete design.

What are the steps? The requirements model is examined to isolate the hierarchical set of problems to be solved. The problem space is partitioned so that subsets of problems associated with specific software functions and features can be identified. Problems can also be organized by type: architectural, component-level, algorithmic, user interface, and so forth. Once a subset of problems is defined, one or more pattern repositories are searched to determine if a design pattern, represented at an appropriate level of abstraction, exists.

What is the work product? A design model that depicts the architectural structure, user interface, and component-level detail is developed.

How do I ensure that I've done it right? As each design pattern is translated into some element of the design model, work products are reviewed for clarity, correctness, completeness, and consistency with requirements and with one another.

But what if the solution were codified in some manner? What if there was a standard way of describing a problem (so you could look it up), and an organized method for representing the solution to the problem? It turns out that software problems have been codified and described using a standardized template, and solutions to them (along with constraints) have been proposed. Called *design patterns,* this codified method for describing problems and their solution allows the software engineering community to capture design knowledge in a way that enables it to be reused.

The early history of software patterns begins not with a computer scientist but a building architect, Christopher Alexander, who recognized that a recurring set of problems was encountered whenever a building was designed. He characterized these recurring problems and their solutions as *patterns,* describing them in the following manner [Ale77]: "Each pattern describes a problem that occurs repeatedly in our environment and then describes the core of the solution to that problem in such a way that you can use the solution a million times over without ever doing it the same way twice." Alexander's ideas were first translated into the software world in books by Gamma [Gam95], Buschmann [Bus07], and their many colleagues.[1] Today, dozens of pattern repositories exist, and pattern-based design can be applied in many different application domains.

14.1 DESIGN PATTERNS

A *design pattern* can be characterized as "a three-part rule which expresses a relation between a certain context, a problem, and a solution" [Ale79]. For software design, *context* allows the reader to understand the environment in which the problem resides and what solution might be appropriate within that environment. A set of requirements, including limitations and constraints, acts as a *system of forces* that influences how the problem can be interpreted within its context and how the solution can be effectively applied.

It is reasonable to argue that most problems have multiple solutions, but that a solution is effective only if it is appropriate within the context of the existing problem. It is the system of forces that causes a designer to choose a specific solution. The intent is to provide a solution that best satisfies the system of forces, even when these forces are contradictory. Finally, every solution has consequences that may have an impact on other aspects of the software and may themselves become part of the system of forces for other problems to be solved within the larger system.

An effective design pattern: (1) captures a specific solution to a bounded problem, (2) provides a solution that has been proven in practice, (3) identifies an approach to a problem that is not obvious, (4) identifies the relationship(s) between the design and other architectural elements, and (5) is elegant in its approach and its utility.

1 Earlier discussions of software patterns do exist, but these two classic books were the first cohesive treatments of the subject.

A design pattern saves you from "reinventing the wheel," or worse, inventing a "new wheel" that is slightly out of round, too small for its intended use, and too narrow for the ground it will roll over. Design patterns, if used effectively, will invariably make you a better software designer.

14.1.1 Kinds of Patterns

One of the reasons that software engineers are interested in (and intrigued by) design patterns is that human beings are inherently good at pattern recognition. If we weren't, we'd be frozen in space and time—unable to learn from experience, unwilling to venture forward because of our inability to recognize situations that might lead to high risk, unhinged by a world that seems to have no regularity or logical consistency. Luckily, none of this occurs because we do recognize patterns in virtually every aspect of our lives.

In the real world, the patterns we recognize are learned over a lifetime of experience. We recognize them instantly and inherently understand what they mean and how they might be used. Some of these patterns provide us with insight into recurring phenomenon. For example, you're on your way home from work on the interstate when your navigation system (or car radio) informs you that a serious accident has occurred on the interstate in the opposing direction. You're 4 miles from the accident, but already you begin to see traffic slowing, recognizing a pattern that we'll call **RubberNecking.** People in the travel lanes moving in your direction are slowing at the sight of the accident to get a better view of what happened on the opposite side of the highway. The **RubberNecking** pattern yields remarkably predictable results (a traffic jam), but it does nothing more than describe a phenomenon. In patterns jargon, it might be called a *nongenerative* pattern because it describes a context and a problem, but it does not provide any clear-cut solution.

When software design patterns are considered, we strive to identify and document *generative* patterns. That is, we identify a pattern that describes an important and repeatable aspect of a system and that provides us with a way to build that aspect within a system of forces that are unique to a given context. In an ideal setting, a collection of generative design patterns could be used to "generate" an application or computer-based system whose architecture enables it to adapt to change. Sometimes called *generativity,* "the successive application of several patterns, each encapsulating its own problem and forces, unfolds a larger solution which emerges indirectly as a result of the smaller solutions" [App00].

Design patterns span a broad spectrum of abstraction and application. *Architectural patterns* describe broad-based design problems that are solved using a structural approach. *Data patterns* describe recurring data-oriented problems and the data modeling solutions that can be used to solve them. *Component patterns* (also referred to as *design patterns*) address problems associated with the development of subsystems and components, the way they communicate with one another, and their placement within a larger architecture. *Interface design patterns* describe common user interface problems and their solutions with a system of forces that includes the specific characteristics of end users. *WebApp patterns* address a problem set that is encountered